

1.3 Gradient Descent

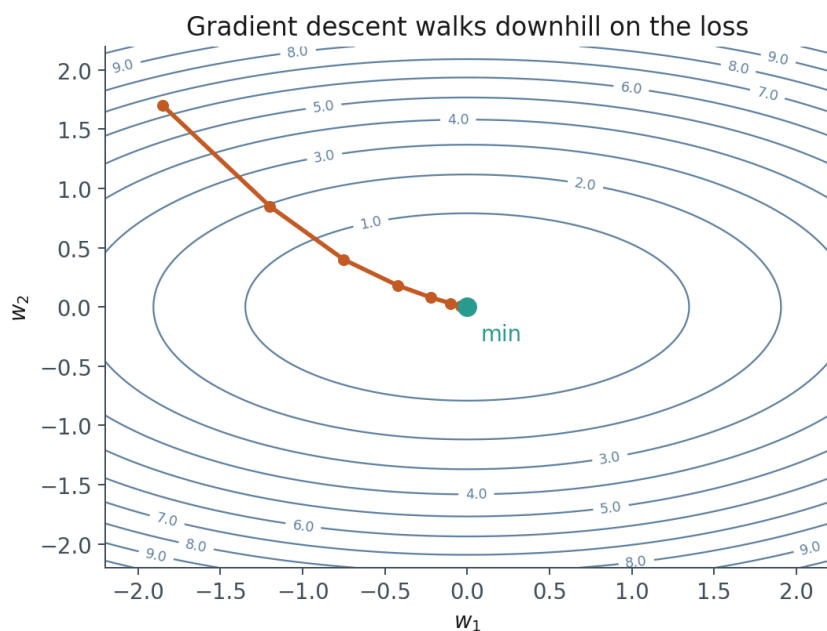
Forward, loss, backward, update. That loop is pretraining and fine-tuning.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Training a net is an optimization problem: pick weights so a **loss** is small on training batches, then check a held-out set. Gradient descent is the default way to move the weights.

1. Loss as a surface

Collect all weights and biases into one vector θ . A loss $L(\theta)$ is a scalar. For classification it is often **cross-entropy**; for a regression toy it is squared error. In two dimensions you can draw the contours. Gradient descent follows $-\nabla L$.



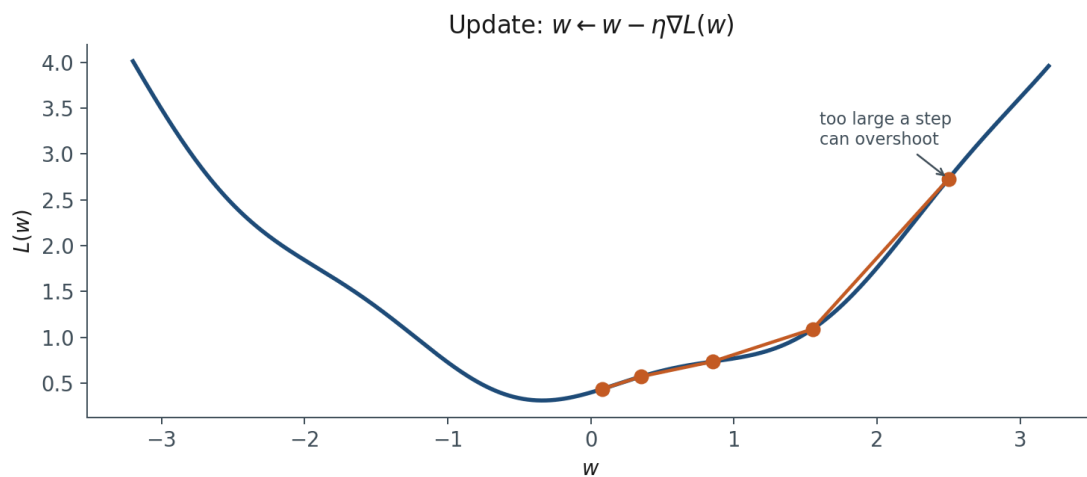
The picture is a cartoon. Real nets live in millions of dimensions. The geometry is still the same idea: the gradient is the direction of steepest increase, so you walk the other way.

2. The update

One step is

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta).$$

η is the **learning rate**. Too small: you crawl. Too large: you jump over the valley or diverge.



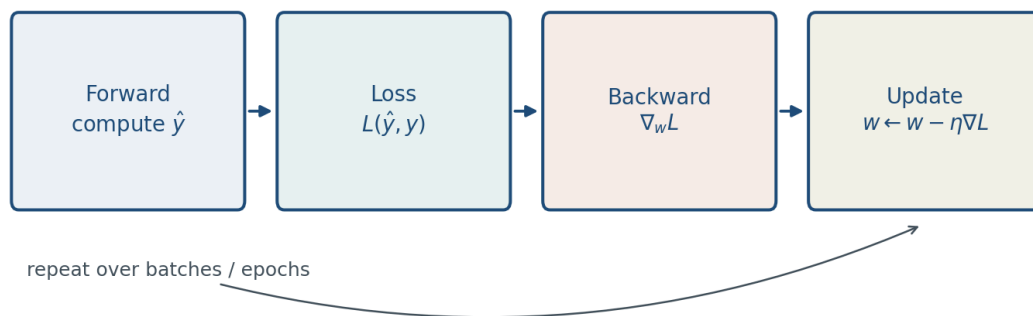
Stochastic gradient descent (SGD) estimates ∇L on a **minibatch**, not the full dataset. That is how you train on Wikipedia-scale text. Adam and related methods rescale coordinates using a running average of gradients; you will use them in PyTorch without deriving them today.

3. Forward, loss, backward, update

One training iteration is four moves:

1. **Forward.** Compute activations layer by layer. Keep them; the backward pass needs them.
2. **Loss.** Compare $\hat{y}$ to the target y .
3. **Backward.** Chain rule from the loss to every weight (**backpropagation**).
4. **Update.** Take a descent step.

Training a net is this loop. Backprop is just the chain rule.



PyTorch does (3) for you if the forward pass used `nn.Module` and a differentiable loss. Your job is to write (1), pick L , and call `optimizer.step()`.

4. What this has to do with language models

Pretraining an LLM is this loop on a next-token loss, at huge batch and data scale. Fine-tuning (Week 8) is the same loop on a smaller, more specific dataset. RLHF (Week 9) changes what L is, not the fact that you are differentiating a scalar with respect to weights.

If the gradient is noise, or η is wrong, or the net is linear, no amount of later “prompt engineering” will save the representation. Get this loop right on a tiny problem in Lab 1 before you touch Hugging Face.

5. Practice

1. You double η and the loss oscillates. What happened geometrically?
2. Why keep $a^{(1)}$ after you already have $\hat{y}$?
3. SGD uses a minibatch. Name one reason that is not “the full gradient is too slow.”